

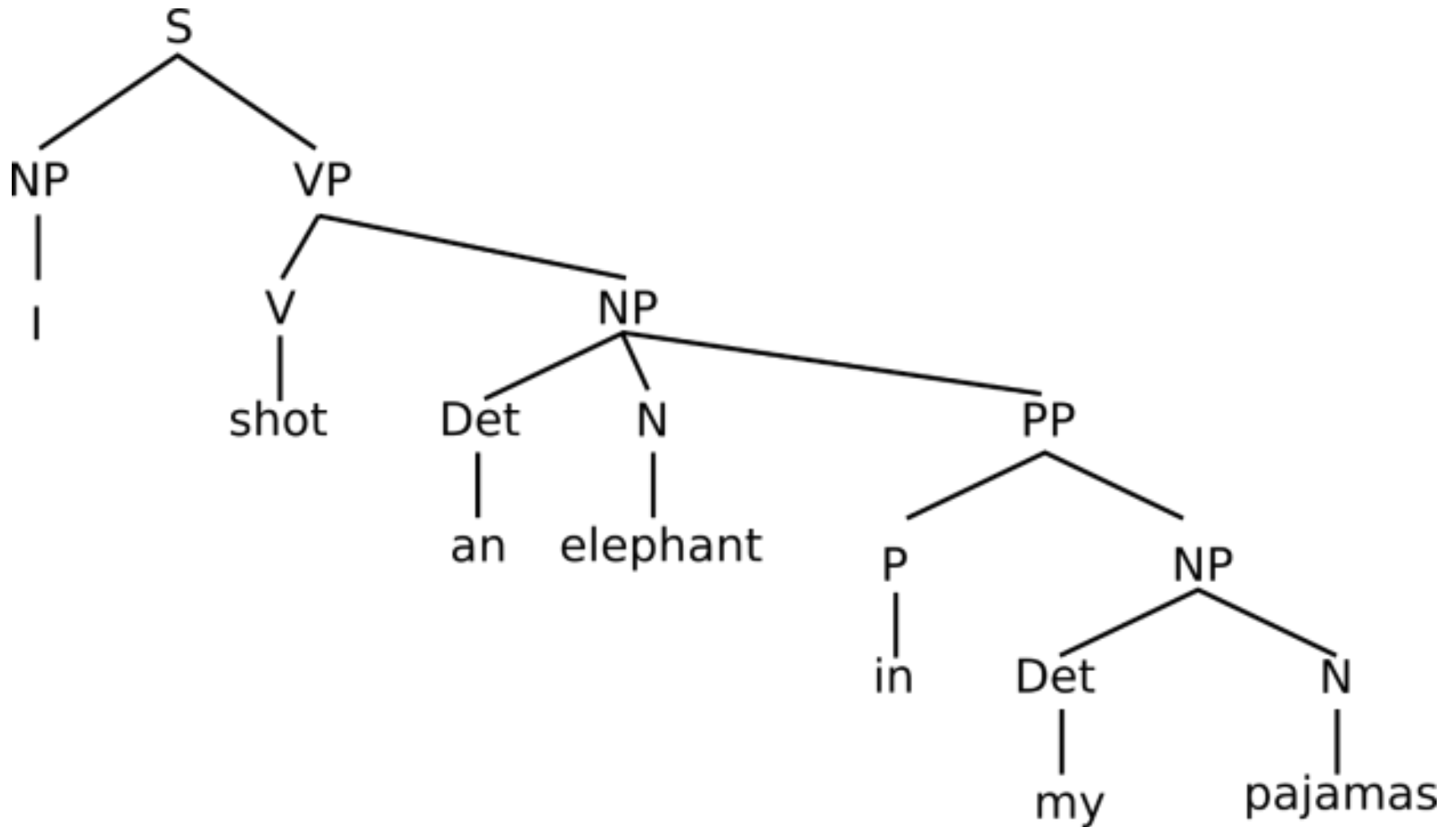
PARSING

- ❖ The context-free grammars used for parsing so far find the constituent-based structure of the input sentence. This is useful for detecting the phenomenon of “omnipresent ambiguity”.

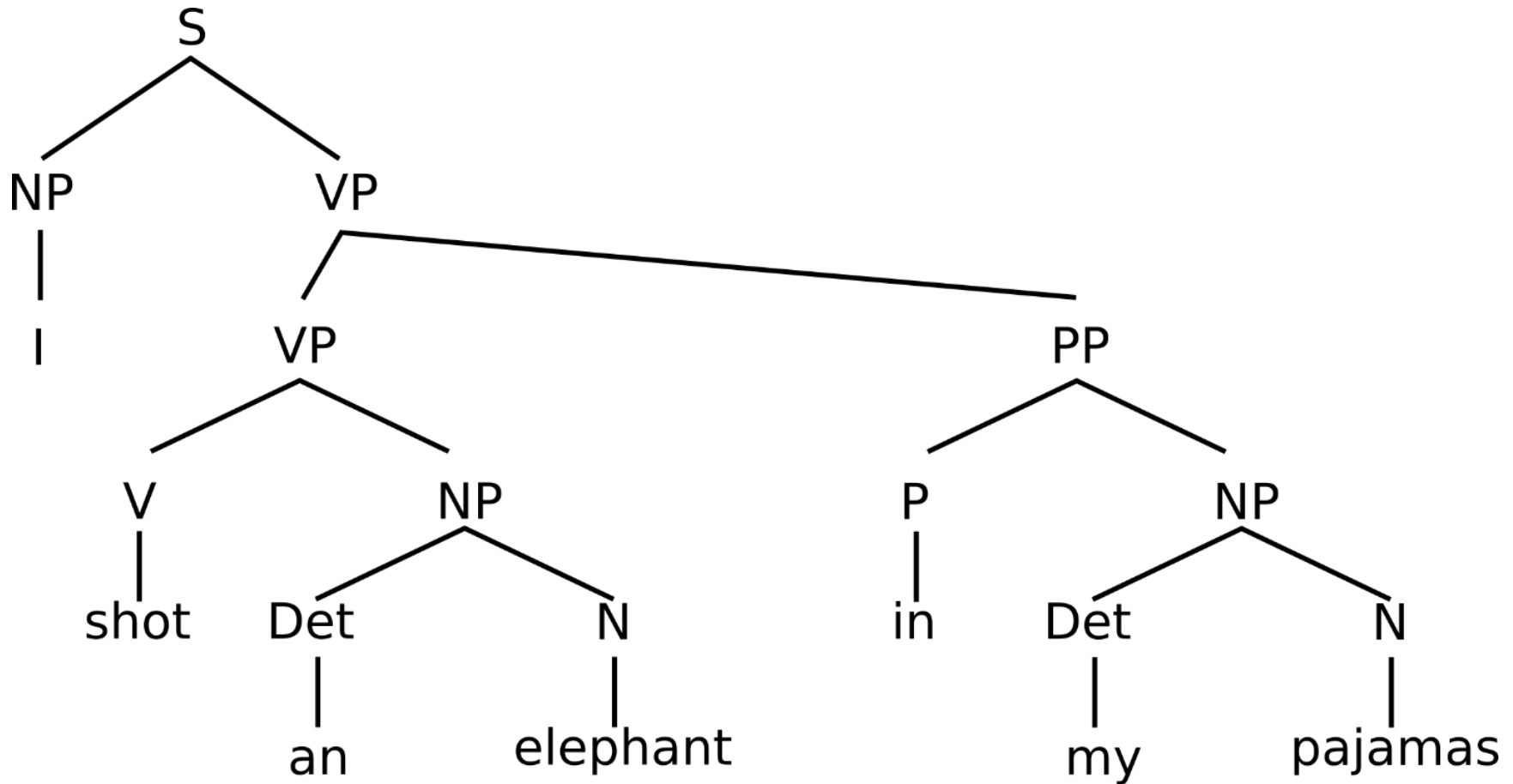
Example: “*While hunting in Africa, I shot an elephant in my pajamas. How an elephant got into my pajamas I’ll never know*”.

- ❖ The ambiguity is reflected by the two resulting parse trees. This is structural ambiguity that reflects the existing semantic ambiguity.

Parse tree no. 1



Parse tree no. 2



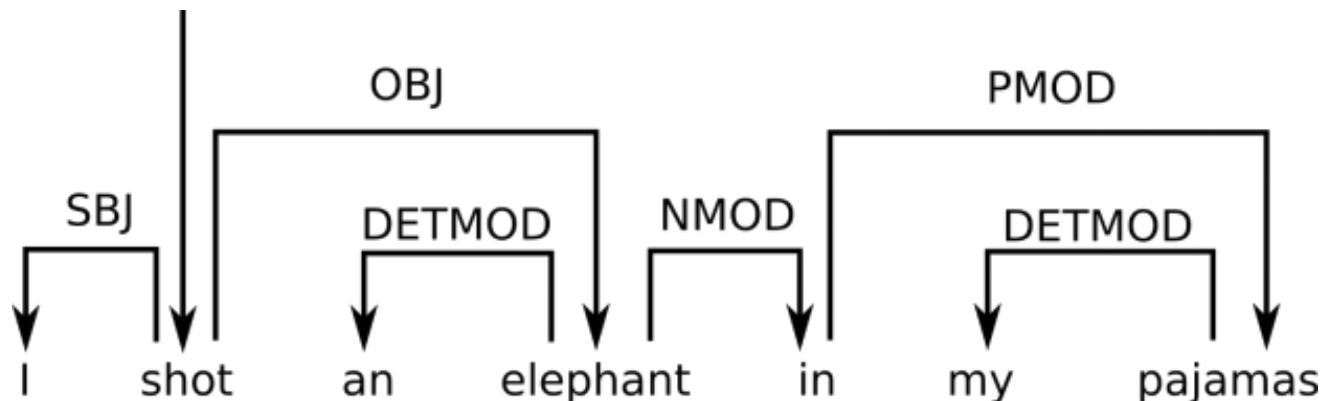
Parse tree no.1 corresponds to the correct sense, which can only be disclosed within the context. Here the elephant is inside the pajamas.

Dependencies and Dependency Grammars

- Context-free grammars (phrase-structure grammars) illustrate how words and sequences of words combine to form constituents.
- The dependency grammar approach focuses instead on how words relate to other words.
- Dependency is a binary asymmetric relation that holds between a head and its dependents.

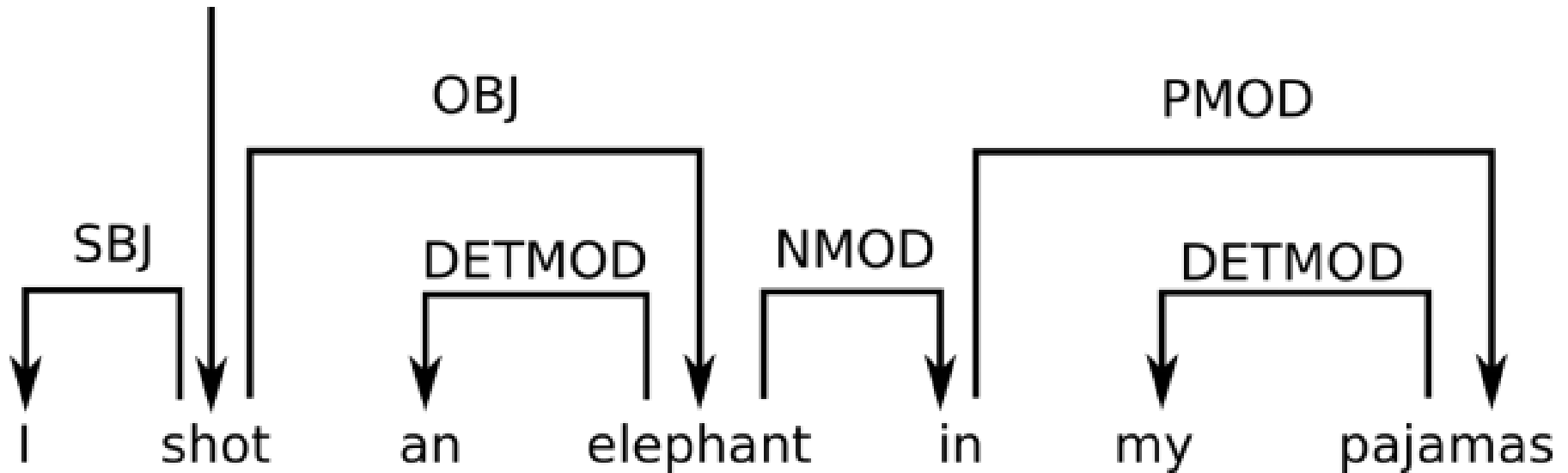
Example

Here is the dependency structure (parse) of the same sentence:



- The head of a sentence is usually taken to be the tensed verb.
- Every other word is either dependent on the sentence head or connects to it through a path of dependencies.
- Dependency grammar (DG) is a class of modern grammatical theories that are all based on the dependency relation (as opposed to the relation of phrase structure) and that can be traced back primarily to the work of Lucien Tesnière.
- Dependency is the notion that linguistic units, e.g. words, are connected to each other by directed links. The tensed verb is taken to be the structural center of clause structure. All other syntactic units (words) are either directly or indirectly connected to the verb in terms of the directed links, which are called *dependencies*.
- A dependency representation of a sentence is a labeled directed graph, where the nodes are the lexical items and the labeled arcs represent dependency relations from heads to dependents.

**Let's look again at the dependency structure of the same sentence
(below):**



Dependency structure:

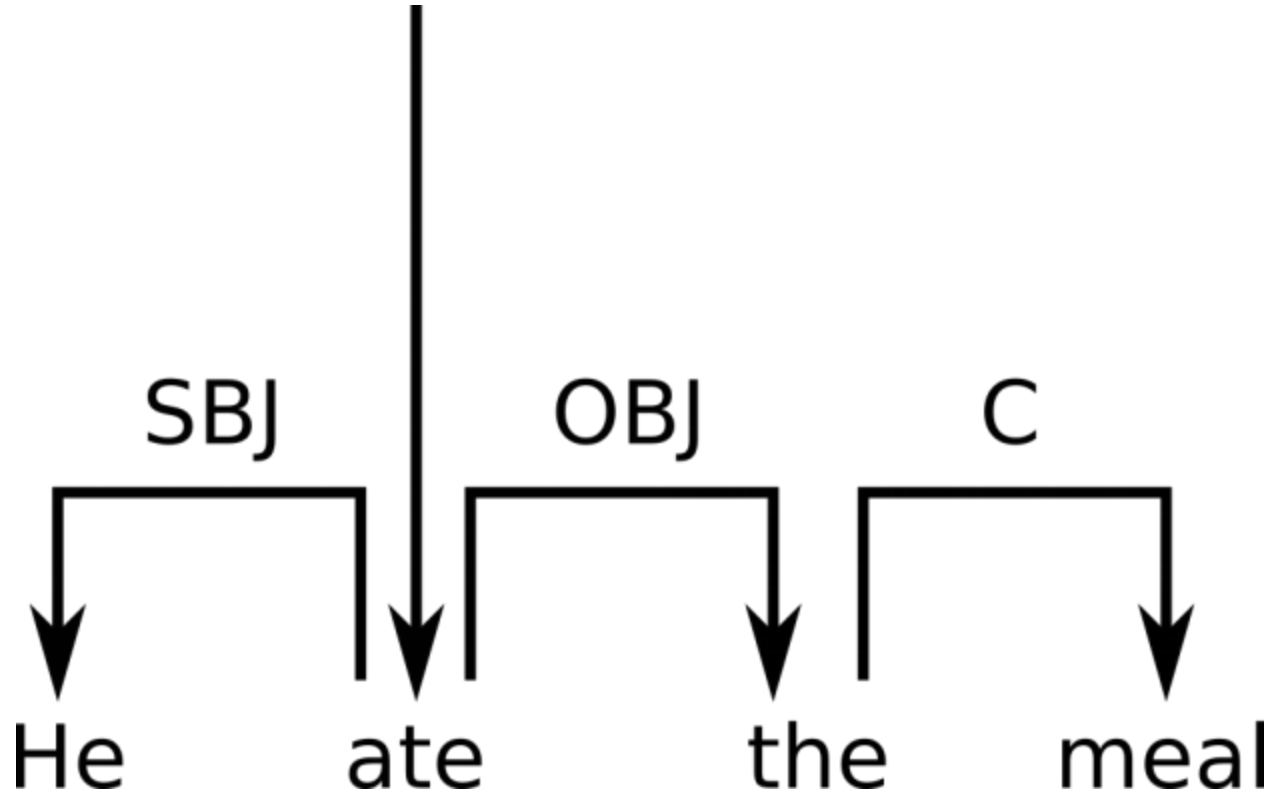
- The arcs are labeled with the grammatical relations between a dependent and its head. For instance:
 - the verb “shot” is the head of the entire sentence;
 - the pronoun “I” is in the grammatical relation SBJ (is the subject of) with the verb “shot”;
 - “in” is in the grammatical relation NMOD (noun modifier) with “elephant”.

These grammatical functions (given by relations) represent the dependency type. (Funcțiile gramaticale sunt exprimate prin tipurile dependente).
- A dependency graph is projective (*projective analysis*) if, when all the words are written in linear order, the edges can be drawn above the words without crossing. This is equivalent to saying that a word and all its descendants (dependents and dependents of its dependents, etc.) form a contiguous sequence of words within the sentence.
- Dependency parsing is non-projective when the edges are allowed to cross.
- The dependency grammar theory has initially considered only the dependency syntactic analysis of projective type.

Main ideas:

- **Dependency grammars are not based on the concept of constituent, but on the direct relations existing among words, viewed as dependency relations.**
- **At the heart of dependency grammars we have the relation existing between the head word and the dependent word.**
- **In this framework, the syntactic analysis of a sentence (*dependency parse*) consists of describing *all* dependency relations existing among *all* words of a sentence.**
- **The dependency graph that represents the dependency parse of a sentence can be projective or non-projective. Initially, only the projective type of analysis was considered. However, in languages with more flexible word order than English, non-projective dependencies are more frequent.**
- **A major characteristic of this type of syntactic analysis is that it views syntactic phrases (noun phrase, verb phrase etc.) as representing byproducts of the existing dependencies! (byproduct = produs secundar)**

Comparison:



COMPARISON:

1. Syntactic analysis based on constituents

- the syntactic phrase “the meal” represents a realization of the phrase-structure rule

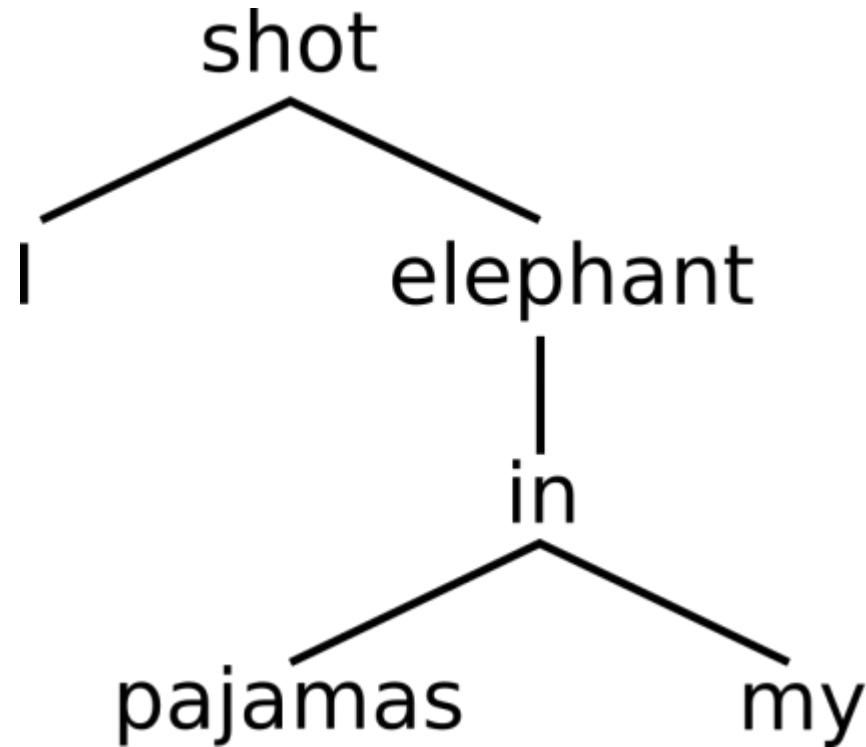
$$NP \rightarrow Det N$$

i.e. it is a noun phrase organized around the noun “meal” – having the head role within the syntactic group

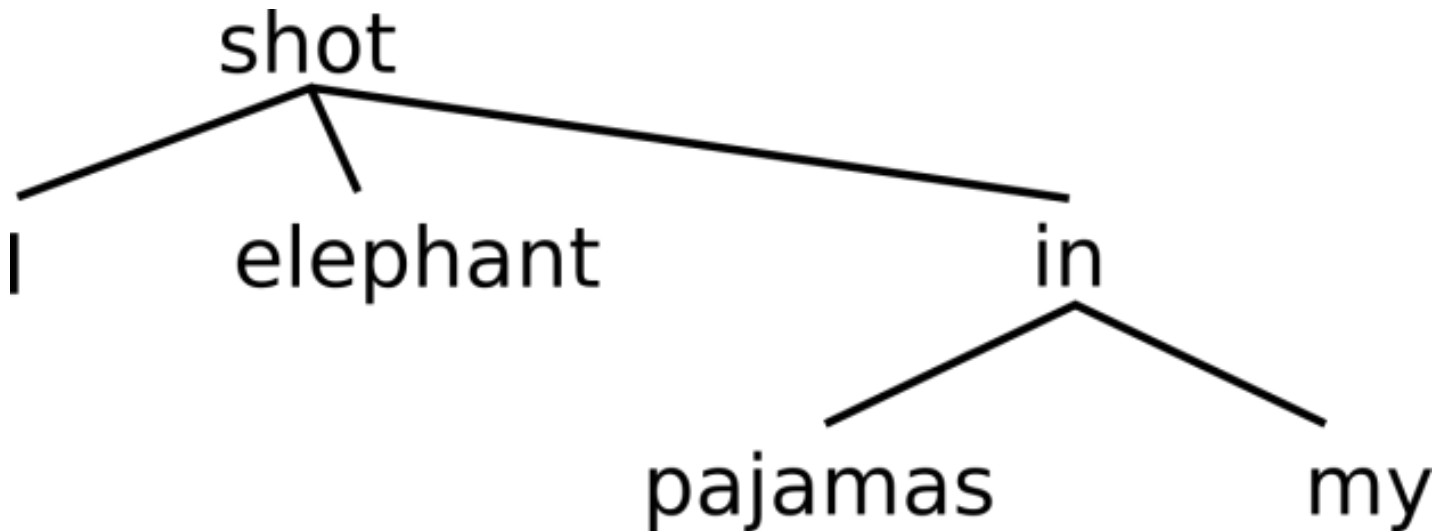
2. Syntactic analysis of dependency type

- “the” represents the head and “meal” is the dependent (because “the” brings in, inserts “meal” into the sentence).

- **Note that:** A great variety of dependency relations may exist among the words of a sentence. The role of dependency grammars is that of specifying the restrictions that the dependency relations must meet in order for the structure they define to be correct from the linguistic point of view.
- The dependency structure of a sentence can also be represented as a tree, one in which the dependents present themselves as sons of the corresponding head words.
- The ambiguity displayed by our previous example leads to two different dependency structures having the following corresponding parse trees:



Here the elephant is inside the pajamas. Note that “in” depends on “elephant”. The analysis is a projective one.



Here I was dressed in my pajamas when I shot the elephant. Note that “in” depends on “shot”. The dependency type is different. The analysis is again a projective one.

CRITERIA for deciding what is the head H and what is the dependent D in a construction C (most important ones):

- 1. H determines the distribution class of C; or alternatively, the external syntactic properties of C are due to H.**
- 2. H determines the semantic type of C.**
- 3. H is obligatory while D may be optional.**
- 4. H selects D and determines whether it is obligatory or optional.**
- 5. The morphological form of D is determined by H (e.g., agreement or case government).**

Main TOOL for dependency parsing:

STANFORD DEPENDENCY PARSER

Stanford dependencies are triplets consisting of:

- name of the relation**
- governor**
- dependent**

See dependency parsing with NLTK at the lab !

REPRESENTING WORDS FOR NLP

There are currently two main ways to represent words in NLP:

- Word embeddings -
 - a distributed representation
- Semantic networks

- ✓ Word embeddings: Word embeddings are a class of techniques where individual words are represented as real-valued vectors in a predefined vector space. Each word is mapped to one vector and the vector values are learned in a way that resembles a neural network (see **Word2vec** with gensim at the lab).
- ✓ Semantic networks: a knowledge base that represents semantic relations between concepts in a network. The semantic network represents an instrument for knowledge representation. It is a directed or undirected graph consisting of vertices, which represent concepts, and edges, which represent semantic relations between concepts (see the next lecture and lab for the semantic network **WordNet**).

Word2vec

- The word2vec algorithm uses a neural network model to learn word associations from a large corpus of text. Once trained, such a model can detect synonymous words or suggest additional words for a partial sentence.
- Word2vec is a group of related models that are used to produce word embeddings. These models are shallow (putin adanc), two-layer neural networks that are trained to reconstruct linguistic contexts of words. Word2vec takes as its input a large corpus of text and produces a vector space, typically of several hundred dimensions, with each unique word in the corpus being assigned a corresponding vector in the space. Word vectors are positioned in the vector space such that words that share common contexts in the corpus are located close to one another in the vector space. The vectors are chosen such that a simple mathematical function (the cosine similarity between the vectors) indicates the level of semantic similarity between the words represented by those vectors.

- In NLP, “word embedding” is a term used for the representation of words for text analysis, typically in the form of a real-valued vector that encodes the meaning of the word such that the words that are closer in the vector space are expected to be similar in meaning.

- Word2vec was created, patented, and published in 2013 by a team of researchers led by Tomas Mikolov at Google over these two papers:
 - ✓ Mikolov, Tomas; et al. (2013). "Efficient Estimation of Word Representations in Vector Space".
 - ✓ Mikolov, Tomas (2013). "Distributed representations of words and phrases and their compositionality". *Advances in Neural Information Processing Systems*.

- The general method (“A Neural Probabilistic Language Model”, 2003):

“Associate with each word in the vocabulary a distributed word feature vector ... The feature vector represents different aspects of the word: each word is associated with a point in a vector space. The number of features ... is much smaller than the size of the vocabulary.”

- Word2vec can utilize either of two model architectures to produce a distributed representation of words: continuous bag-of-words (CBOW) or continuous skip-gram.

- In the continuous bag-of-words (CBOW) architecture, the model predicts the current word from a window of surrounding context words. The order of context words does not influence prediction (bag-of-words assumption)
 - In the continuous skip-gram architecture, the model uses the current word to predict the surrounding window of context words. The skip-gram architecture weighs nearby context words more heavily than more distant context words.
- ✓ Skip-gram: works well with a small amount of the training data, represents well even rare words or phrases.
 - ✓ CBOW: several times faster to train than the skip-gram, slightly better accuracy for the frequent words.
 - ✓ Note that results of word2vec training can be sensitive to parametrization (see the lab). The use of different model parameters and different corpus sizes can greatly affect the quality of a word2vec model.

❖ The word embedding approach is able to capture multiple different degrees of similarity between words. Mikolov et al. (2013) found that semantic and syntactic patterns can be reproduced using vector arithmetic. Patterns such as “*Man is to Woman as Brother is to Sister*” can be generated through algebraic operations on the vector representations of these words such that the vector representation of “*Brother*” - “*Man*” + “*Woman*” produces a result which is closest to the vector representation of “*Sister*” in the model. Such relationships can be generated for a range of semantic relations (such as Country–Capital) as well as syntactic relations (e.g. present tense–past tense).

Implementation – with Gensim

- Gensim is a Python library for *topic modelling*, *document indexing* and *similarity retrieval* with large corpora.
- There are several ways to train word vectors in Gensim among which is Word2Vec.
- **models.word2vec** – Word2vec embedding
This module implements the word2vec family of algorithms. Tutorial:
<https://rare-technologies.com/word2vec-tutorial/>

Different Text Representation Techniques

Technique	Main Characteristics	Use Cases
BOW / TF-IDF	Statistical method to capture the relevance of words w.r.t the corpus of text. It does not capture semantic word associations.	Better for information retrieval and keyword extraction in documents.
Word2Vec	Neural network-based CBOW and Skip-gram architectures, better at capturing semantic information.	Useful in semantic analysis task.
GLOVE	Matrix factorization based on global word-word co-occurrence. It solves the local context limitations of Word2Vec.	Better at word analogy and named-entity recognition tasks. Comparable results with Word2Vec in some semantic analysis tasks while better in others.
Transformer	Transformer-based attention mechanism to capture high-quality contextual information.	Language translation, question-answering system. Deployed in Google Search engine to understand search queries.